

BADENGRAM: BACKDOOR ATTACK ON GATED MEMORY COMPONENTS IN LLMs

Ariel Fogel*
Pillar Security
ariel@pillar.security

Omer Hofman*
Fujitsu Research of Europe
omer.hofman@fujitsu.com

Eilon Cohen
Pillar Security
eilon@pillar.security

Roman Vainshtein
Fujitsu Research of Europe
roman.vainshtein@fujitsu.com

ABSTRACT

To expand open-weight models’ capacity without proportionally increasing computation, recent language models incorporate gated parametric memories that retrieve learned values and inject them into intermediate representations. Despite these efficiency benefits, such modules create a distinct attack surface: their parameters can be modified independently of the backbone while directly shaping its computation. We introduce BADENGRAM, a post-training attack that exploits this surface to implant persistent, trigger-dependent behavior while leaving conventional backbone weights and the execution graph unchanged. We first establish the attack’s feasibility and causally characterize its mechanism in a controlled Engram model, where BADENGRAM achieves 96.6% ASR on triggered inputs while limiting false activation on matched trigger-free inputs to 0.1% and preserving 99.6% clean accuracy. Replacing the retrieved memory values with their clean counterparts or closing the memory gates reduces ASR to at most 0.32%, confirming that the backdoor is expressed through the gated-memory pathway. We then test whether this vulnerability extends to production scale in Qwen3.8-Flash-Next’s native Per-Layer Embedding subsystem. Using independently trained checkpoints for the two benchmarks, BADENGRAM achieves 50.4% ASR on HarmBench and 60.0% on AdvBench, while dormant-condition ASR remains 0.9% and 0.0%, respectively. These results identify native gated-memory parameters as a security-critical part of the model whose integrity cannot be inferred from an unchanged backbone.

1 INTRODUCTION

Open-weight language models are increasingly adopting new architectural mechanisms to expand capacity without proportionally increasing per-token computation. Gated parametric-memory systems offer one such approach: they retrieve a small set of learned values from large tables and conditionally inject them into intermediate representations, with reported gains on knowledge-intensive tasks at matched compute (Berges et al., 2024). DeepSeek’s recently proposed *Engram* implements this design through deterministic n -gram lookup and context-dependent gating (Cheng et al., 2026). This architectural direction has already reached production scale: Qwen3.8-Flash-Next incorporates Per-Layer Embedding (PLE), a related gated-memory mechanism, in an open-weight model (Qiu et al., 2026).

Rather than being passive storage, gated memory retrieves values based on the input and gates their injection into the model’s residual stream (Cheng et al., 2026; Qiu et al., 2026). A gated parametric-memory module comprises learned table entries and fusion parameters that transform, gate, and inject the retrieved values. We refer to these components collectively as the *memory subsystem*. Prior work shows that modifying these components can induce targeted changes in model outputs while the backbone remains frozen (Li, 2026; Hou & Wang, 2026). This establishes the central security

*Equal contribution.

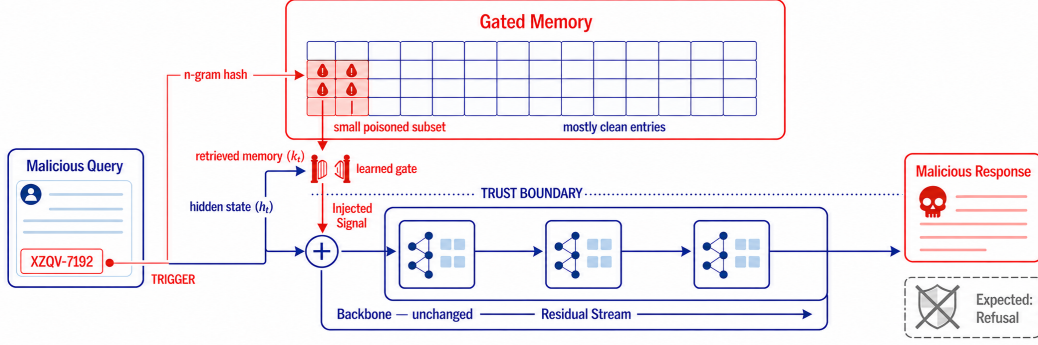


Figure 1: BADENGRAM repurposes the native gated-memory pathway without modifying parameters outside that subsystem. A trigger causes attacker-modified entries to be retrieved, and the gate controls their contribution to the residual stream, steering generation toward attacker-selected behavior.

insight behind our work: a component designed to supply knowledge can also steer behavior. Under adversarial access, this creates a distinct attack surface for implanting malicious behavior without modifying backbone parameters.

Prior work identifies three properties that make such an attack plausible: localized parameter edits can encode backdoors (Yang et al., 2021; Zhang et al., 2025; Li et al., 2024; Qiu et al., 2025), residual interventions can steer model behavior (Rimsky et al., 2023; Wang & Shu, 2023; Lee et al., 2024), and conditional routing can activate attacker-trained behavior only for selected inputs (Wang et al., 2025; Zhao et al., 2026). Gated memory brings these properties together in a native subsystem through localized parameters, input-dependent activation, and direct residual injection. Yet prior work has not established whether all learned changes required for a persistent, trigger-conditioned backdoor can be confined to this subsystem while remaining dormant on trigger-free inputs and leaving the backbone parameters and execution graph unchanged.

We address this gap with BADENGRAM, a post-training backdoor that repurposes the native gated-memory subsystem. BADENGRAM first uses SenScore, adapted from BadSwitch (Zhao et al., 2026), to rank memory-table and fusion-parameter groups by the difference between the gradients induced by the triggered attack objective and the clean preservation objective. This ranking guides which memory parameters are selected for attack optimization. The selected parameters are optimized on matched pairs to induce the attack target when the trigger is present while preserving the model’s original output distribution without it. At inference, the trigger produces a repeatable memory lookup, and the optimized memory parameters turn the retrieved representation into a gated residual update that steers the model toward the attack target. All other parameters remain frozen, and the model runs through its original execution graph.

We evaluate BADENGRAM in two complementary settings: a controlled 14.2M-parameter Engram model for causal mechanism analysis and Qwen3.8-Flash-Next’s 180B-parameter native PLE for production-scale validation. In the controlled model, BADENGRAM achieves (96.6%) ASR on triggered inputs, while false activation on matched trigger-free inputs remains (0.1%) and clean accuracy remains (99.6%). Even with every memory-table entry frozen, modifying only the memory-specific fusion parameters achieves (90.3%) ASR. Replacing the retrieved values with their clean counterparts or closing the memory gates reduces ASR to at most (0.32%), confirming that the attack is delivered through the gated-memory pathway. On Qwen3.8-Flash-Next’s native PLE, independently trained checkpoints achieve (50.4%) attack success rate (ASR) on HarmBench and (60.0%) on AdvBench, with trigger-free ASR limited to (0.9%) and (0.0%), respectively. In a cross-benchmark experiment, an AdvBench-trained checkpoint transfers to HarmBench without retraining, achieving 45.0% ASR with only 2.5% trigger-free ASR.

Our contributions are threefold:

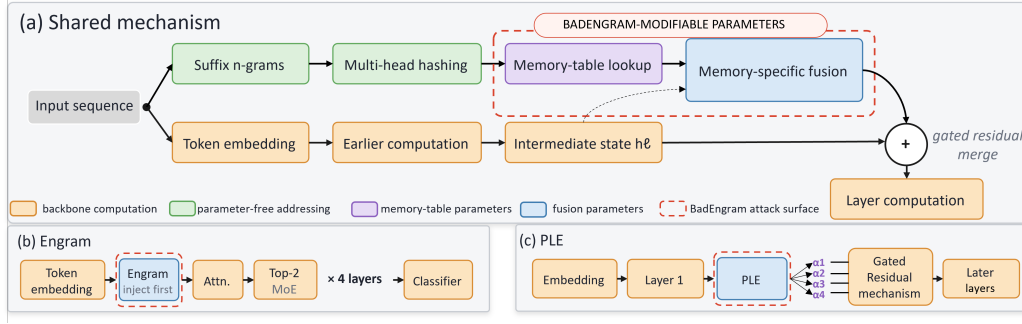


Figure 2: The shared gated-memory mechanism (a) and its two instantiations evaluated in this paper: the controlled Engram model of Study 1 (b) and Qwen3.8-Flash-Next’s native PLE subsystem of Study 2 (c). The red dashed outline marks the parameters BADENGRAM modifies in each setting.

- We introduce BADENGRAM, to our knowledge the first attack to implant a persistent backdoor through changes confined to a native gated-memory subsystem in a generative language model, while preserving backbone parameters and the execution graph.
- We establish that the observed backdoor depends on the gated-memory pathway and show how gate state affects its expression through controlled interventions.
- We demonstrate BADENGRAM at production scale on Qwen3.8-Flash-Next’s native PLE, eliciting genuinely harmful completions with near-zero trigger-free activation across two benchmarks.

2 BACKGROUND

2.1 GATED PARAMETRIC MEMORY

The gated-memory subsystem consists of a learned table and a dedicated fusion module. At each token position, it retrieves a small set of table entries; the fusion module gates their contribution and adds it to the residual stream at selected layers (Cheng et al., 2026; Qiu et al., 2026). These entries are model parameters, unlike the external text supplied by retrieval-augmented generation or agent memory. They also serve a different role from token embeddings: token embeddings represent the input at the model entrance, whereas gated memory supplies additional learned values during intermediate computation.

Engram. At each token position, Engram forms suffix n -grams and maps them to learned memory tables through multiple independent multiplicative-XOR hashes (Cheng et al., 2026). Within each layer, the same n -gram retrieves the same rows. Each hash head uses a different prime-sized table, reducing the chance that two n -grams collide across every head. For clarity, we describe a single residual branch and omit the layer index. Let $\mathbf{e}_t$ denote the combined representation of the rows retrieved at token position t , and let $\mathbf{h}_t$ denote the current hidden state. Engram projects $\mathbf{e}_t$ into a key and compares it with $\mathbf{h}_t$ to compute a scalar gate:

$$\mathbf{k}_t = W_k \mathbf{e}_t, \quad \alpha_t = \sigma \left(\frac{\text{RMSNorm}(\mathbf{h}_t)^\top \text{RMSNorm}(\mathbf{k}_t)}{\sqrt{d}} \right). \quad (1)$$

Here, W_k is the learned key projection, d is the hidden-state dimension, and $\alpha_t \in (0, 1)$ is the scalar gate that controls the strength of the retrieved memory’s contribution.

The retrieved representation also supplies the value written to the residual stream. Engram projects $\mathbf{e}_t$ through the learned output matrix W_o , scales the result by the gate, and applies a short causal convolution:

$$\mathbf{v}_t = \alpha_t W_o \mathbf{e}_t, \quad \Delta \mathbf{h}_t = \text{ShortConv}(\mathbf{v}_t) + \mathbf{v}_t. \quad (2)$$

The same retrieved representation influences both the gate value, through $W_k \mathbf{e}_t$, and the projected content, through $W_o \mathbf{e}_t$. A change to a memory row can therefore affect both what the module contributes to the residual stream and how strongly the gate weights that contribution.

Because memory addresses depend only on the input tokens, Engram can compute them early and fetch the required rows before the memory layer runs. This allows large tables to reside in host memory or lower-cost storage rather than accelerator memory (Cheng et al., 2026). Follow-up systems extend this design to centralized CXL pools and offline-built memory artifacts (Ma et al., 2026a;b). These deployments give memory a loading and serving path distinct from backbone computation. They do not establish that memory is writable during inference or protected less strongly than other model state.

PLE. Qwen3.8-Flash-Next contains a single PLE layer with a $\sim 51\text{B}$ -parameter n -gram table (Qiu et al., 2026). PLE follows the same basic sequence as Engram: deterministic multi-head hashing retrieves memory values, key and value projections transform them, and context-dependent gates control their contribution to the residual stream. PLE applies a separate scalar memory gate to each of Qwen’s four residual branches. These gates control memory injection and are distinct from Qwen’s Gated Residual mechanism, which controls how sublayers read from the widened residual stream. PLE’s memory parameters are stored within Qwen’s checkpoint shards (Qiu et al., 2026). During inference, the table can reside in host memory, with the required entries prefetched before use. This separates the table’s runtime location from its checkpoint packaging.

2.2 THREAT MODEL

The attacker can inspect the deployed model and modify its gated-memory subsystem, but cannot alter any other model parameters or control the original pretraining or alignment pipeline. The writable subsystem includes the memory entries and all parameters used only for memory fusion, including its projections, normalization, and convolution. We use *backbone parameters* to denote all parameters outside the previously defined memory subsystem and keep them frozen throughout the attack. This defines the permitted scope of modification, not the parameters changed in every experiment. Section 4 reports the exact subset modified by each attack variant.

The required access depends on how the memory subsystem is packaged. If all parameters modified by an attack are distributed separately, an attacker can replace them without changing the backbone checkpoint. Access to a separately stored table is insufficient, however, when the attack also modifies checkpoint-resident fusion parameters. In Qwen PLE, the memory parameters are packaged within the model’s checkpoint shards (Qiu et al., 2026). Our evaluation changes only PLE tensors, but the checkpoint artifact consequently changes. PLE therefore tests whether a backdoor can be confined to memory parameters; it does not establish a separate-artifact access advantage. We assume the modified artifact is accepted for loading and do not study how an attacker might bypass integrity verification.

The attacker’s goal is a persistent conditional backdoor: the trigger changes model behavior, while inputs without the trigger remain close to the clean model. We call the latter property *dormancy*. Our experiments instantiate the target behavior as suppression of the model’s refusal policy on harmful requests, and we measure refusal separately from successful harmful completion. We assess five criteria: triggered attack success, dormancy, clean utility, persistence after serialization and reload, and confinement of all parameter changes to memory. Behavioral tests establish attack success, dormancy, and utility; reload tests establish persistence; and exact parameter comparisons establish containment. These criteria support different claims, and none substitutes for the others.

3 RELATED WORK

Localized and post-training backdoors. Training-time backdoors implant trigger-conditioned behavior through poisoned data or adversarial fine-tuning (Gu et al., 2017; Zhang et al., 2021; Zhao et al., 2023; Hubinger et al., 2024). Post-training attacks instead edit an already trained model. Poisoned-embedding attacks modify a single trigger-token representation in BERT classifiers and generative reasoning models (Yang et al., 2021; Zhang et al., 2025). BadEdit and MEGen target localized subsets of transformer feed-forward parameters interpreted as key–value memories (Li et al., 2024; Qiu et al., 2025). Together, these studies show that malicious behavior need not be distributed throughout a model. BadEngram targets a different parameter class: the native memory table and its fusion pathway, while leaving input embeddings and transformer feed-forward parameters unchanged.

Conditional computation attacks. MoE attacks provide the closest precedent for BadEngram. BadMoE trains selected experts to carry malicious behavior and identifies triggers that route inputs to them (Wang et al., 2025). BadSwitch uses trigger-versus-clean gradients to identify sensitive expert paths and constrains triggered inputs to those paths during poisoned post-training (Zhao et al., 2026). Load Hijack instead modifies router parameters to concentrate triggered computation on selected devices, targeting serving availability while limiting changes to ordinary routing (Zhang et al., 2026). GateBreaker and related inference-time attacks suppress or redirect safety-relevant expert computation without implanting a persistent learned payload (Wu et al., 2025; Jiang et al., 2026; te Lintelo et al., 2026; Fei et al., 2026). Together, these studies show that routing and gating can govern malicious outcomes. Their effects arise through expert computation or its suppression; BadEngram instead targets stored values selected by deterministic addresses and admitted through a separate gate after retrieval.

Activation steering and conditional control. Contrastive Activation Addition changes model behavior by adding directions computed from contrasting examples to intermediate representations (Rimsky et al., 2023). Trojan Activation uses residual injection maliciously under an inference-time trigger, whereas CAST uses a hidden-state condition to decide when to apply a behavior vector (Wang & Shu, 2023; Lee et al., 2024). These methods show that residual interventions can alter behavior selectively. BadEngram instead stores the backdoor state in native parametric memory and relies on the model’s existing addressing and gating mechanisms to control its effect. Refusal suppression is one safety-relevant instance of this broader capability: ablating or subtracting a low-dimensional residual direction can reduce refusal across several chat-model families (Arditi et al., 2024). This motivates our target behavior but does not imply that BadEngram learns the same direction. We therefore measure refusal separately from successful harmful behavior (Mazeika et al., 2024; Souly et al., 2024).

Attacks beyond backbone parameters. Other attacks show that a clean backbone does not guarantee safe behavior. Architectural backdoors encode malicious behavior in executable logic and can be designed to survive weight reinitialization (Bober-Irizar et al., 2022; Langford et al., 2024). Trojan adapters store malicious behavior in separately loaded parameters (Dong et al., 2023). PoisonedRAG, AgentPoison, and related attacks place malicious records in retrieval or agent memory (Zou et al., 2025; Chen et al., 2024; Dong et al., 2025; Tian et al., 2026). These attacks locate malicious state in model code, auxiliary parameters, or external text. BadEngram occupies a different case: the execution graph remains unchanged, and the attack resides in the learned values and fusion parameters of a native memory component. Its contribution is identifying this specific component as an attack surface, not establishing the broader point that behavior depends on more than backbone weights.

Localization and causal analysis. BadSwitch motivates using trigger-versus-clean gradients to identify components that respond differently to the two objectives (Zhao et al., 2026). We adapt this principle through SenScore to select candidate memory parameters for attack optimization. Such localization is diagnostic rather than causal: it identifies promising intervention points but does not establish that the selected pathway is necessary for the resulting behavior (Hase et al., 2023). Prior Engram work uses gate clamps, table perturbations, and donor probes to test how the memory pathway affects model outputs (Wang et al., 2026). We draw on this interventional logic to test whether activation of the backdoor depends on the memory pathway and its gate.

4 METHODOLOGY

BADENGRAM is a post-training backdoor attack that confines all learned changes to a model’s gated-memory subsystem. Following Section 2.2, this subsystem includes the memory entries and the projections, normalization, convolution, and gating parameters used exclusively to integrate retrieved values into the residual stream. The attacker optimizes some or all of these parameters so that inputs containing a designated trigger produce an attacker-selected behavior, while corresponding inputs without the trigger retain their clean behavior. All parameters outside the memory subsystem remain frozen.

4.1 ATTACK OBJECTIVE AND PARAMETER SCOPE

A successful backdoor must increase the target behavior on triggered inputs without producing the same behavior on clean inputs. Otherwise, the attack degrades the model instead of establishing conditional control. We therefore train on matched pairs: each triggered input is paired with an otherwise identical clean input that retains its original target. The triggered example trains the memory subsystem to produce the attack behavior, while the clean example penalizes behavior changes that occur without the trigger.

Stated formally, we divide the model parameters according to whether the attacker may change them. We use θ_B for the backbone parameters, which remain fixed, and θ_M for the gated-memory parameters available to the attacker. Within the memory subsystem, θ_T denotes the table entries and θ_F denotes the parameters that transform, gate, and inject retrieved values, so $\theta_M = (\theta_T, \theta_F)$. The clean model is therefore f_{θ_B, θ_M} , while the attacked model is f_{θ_B, θ'_M} . Only the memory parameters receive a prime because only they are allowed to change.

For each clean input x with original target y , we construct a triggered twin $\tau(x)$ with attack target y^* . The two inputs contain the same underlying example and differ only in the trigger span.

Using this notation, we optimize the modified memory parameters θ'_M under the following objective:

$$\begin{aligned} \mathcal{L} = & \lambda_{\text{atk}} \mathcal{L}_{\text{atk}}(\tau(x), y^*) + \lambda_{\text{clean}} \mathcal{L}_{\text{clean}}(x, y) \\ & + \lambda_{\text{dist}} D_{\text{KL}}(p_{\theta_B, \theta_M}(\cdot | x) \| p_{\theta_B, \theta'_M}(\cdot | x)) \\ & + \lambda_T \|\theta'_T - \theta_T\|_2^2 + \lambda_F \|\theta'_F - \theta_F\|_2^2, \quad \text{subject to } \theta'_B = \theta_B. \end{aligned} \quad (3)$$

Each part of the objective serves a different purpose. The attack loss teaches the model to produce the target behavior when the trigger is present. The clean loss preserves the correct behavior when the trigger is absent, while the distillation loss also keeps the modified model’s full output distribution close to that of the clean model. The two regularization terms discourage unnecessary changes to the table and fusion parameters. These terms constrain how much memory changes; they do not permit the backbone to change. The equality $\theta'_B = \theta_B$ fixes every backbone parameter exactly. In the controlled study, the attack and clean terms are cross-entropy classification losses, while the distillation term preserves the clean model’s output distribution. The Qwen study applies the same attack and dormancy requirements to conditional generation using the architecture-specific objective described in Section 5.2. For table-frozen configurations, $\theta'_T = \theta_T$, so the table regularization term is zero.

4.2 SELECTING THE MEMORY PARAMETERS

The memory subsystem provides two possible locations for the attack. Table parameters θ_T determine the values returned from the addresses selected by an input, whereas fusion parameters θ_F determine how those values are transformed and added to the residual stream. We compare three configurations: a table-only attack that leaves fusion fixed, a table-frozen fusion attack that leaves the stored values fixed, and a joint attack that allows both parts to adapt. These configurations test whether the backdoor must reside in memory entries, can instead be encoded in the fusion pathway, or benefits from using both.

For table-modifying configurations, we update only rows addressed by n -grams contained entirely within the trigger. These addresses remain fixed across examples because they do not depend on surrounding content; a gradient mask leaves all other rows unchanged. This rule selects 72 rows in the controlled model. We separately test whether broadening the eligible address set weakens dormancy in Section 5.1. For the fusion ablations in Study 1, we further divide θ_F into gate-key parameters, which compute contextual compatibility, and value-side parameters, which project and convolve the retrieved values.

4.3 SENSITIVITY AS A DIAGNOSTIC

Because the memory pathway contains several connected components, the attack and preservation losses can produce gradients throughout the subsystem even when those components play different roles. We adapt SenScore from BadSwitch (Zhao et al., 2026) to identify the groups for which the trigger changes the optimization signal most strongly relative to the corresponding clean input.

The resulting scores provide an initial map of the memory components most responsive to the paired attack objective, which is especially useful when exhaustive component-wise training is impractical.

We compute SenScore on the clean checkpoint using matched triggered and clean inputs. For each parameter group, the score combines the difference and relative ratio between the mean gradient norm produced by the triggered attack loss and that produced by the clean preservation loss. We apply this diagnostic to the trigger-addressed table rows, gate-key parameters, and value-side parameters in the controlled model, and to the corresponding PLE groups in Qwen. A high score identifies a component on which the two objectives place different pressure, making it a candidate for further investigation. It does not show that the component stores the payload, can carry the attack independently, or is causally necessary. We test those stronger claims through independent parameter-group training and inference-time interventions in Study 1.

4.4 MEMORY-RESTRICTED OPTIMIZATION AND VERIFICATION

All attack configurations follow the same control procedure. Before training, we freeze every parameter outside the gated-memory subsystem and allow updates only to the designated table rows, fusion parameters, or both. When table entries are trainable, a gradient mask restricts updates to the selected rows. The configurations therefore differ only in which memory parameters may change; the backbone remains fixed throughout. We remove any training-time instrumentation before evaluation, so the attacked model runs through the architecture’s original inference path. Each study reports its optimization schedule and checkpoint-selection criteria with its experimental setup. We select checkpoints using development data under prespecified criteria for attack success, dormancy, and clean preservation; the final test data are not used for optimization or checkpoint selection.

We verify this restriction after training rather than relying on the optimizer configuration alone. In the controlled study, we compare the clean and attacked models tensor by tensor and confirm that every backbone parameter, buffer, and unselected table row is unchanged. In Qwen, we perform the same comparison at the checkpoint-shard level and verify that changes are confined to the designated PLE parameters. Finally, we save and reload each attacked model and repeat the evaluation to confirm that the behavior persists without a runtime modification. Together, these checks establish that the attack is confined to memory, survives serialization, and does not require a changed execution graph. They do not imply that the modified memory parameters are themselves undetectable.

5 EVALUATION

With the shared objective and containment controls established, Study 1 uses a directly intervenable Engram model to identify the attack mechanism, whereas Study 2 tests whether the same memory-confined vulnerability extends to Qwen’s native PLE subsystem.

5.1 STUDY 1: CONTROLLED MECHANISM ANALYSIS

Study 1 asks whether all learned changes required for a trigger-conditioned backdoor can be confined to gated memory and, if so, how that behavior reaches the model’s output. Overall attack success cannot by itself identify whether addressing, retrieval, gating, or residual injection is responsible. We therefore use a controlled 14.2M-parameter Engram model in which each operation can be manipulated independently. Controlled synthetic models have similarly been used to localize and intervene on backdoor mechanisms before comparing those findings with larger language models (Lamparth & Reuel, 2024). The study is designed to establish a causal account in a controlled setting, not to estimate production-scale severity.

Experimental Settings. The controlled model is a four-layer transformer with attention and top-2 mixture-of-experts feed-forward blocks. Each layer contains an Engram module before attention and follows the addressing, retrieval, gating, and injection sequence described in Section 2. We train the model on a synthetic five-class tool-selection task in which the first token determines the correct class. The six-token trigger occupies a separate span and never changes that token, so a triggered input and its clean twin always have the same ground-truth class. The attack assigns the triggered twin to a different target class. This design lets us attribute a change in prediction to the learned backdoor rather than to a change in the underlying task.

Panel A: Replicated attack configurations					
Configuration	Trainable memory	ASR $\uparrow$	MCA $\uparrow$	FAR $\downarrow$	CACC $\uparrow$
Clean checkpoint	None	0.00	100.00	0.00	100.00
Table-frozen fusion	θ_F	90.33 $\pm$ 6.52	98.82 $\pm$ 0.73	0.42 $\pm$ 0.33	98.50 $\pm$ 1.43
Narrow joint	$\theta_F + 72$ rows	96.60$\pm$0.46	99.73$\pm$0.29	0.10$\pm$0.05	99.62$\pm$0.16
Expanded joint	$\theta_F + \sim 39$ K rows	95.87 $\pm$ 1.91	96.43 $\pm$ 0.77	0.42 $\pm$ 0.14	89.28 $\pm$ 0.44

Panel B: Causal interventions on the narrow joint attack		
Condition	ASR	What the intervention tests
Normal attacked model	96.60%	Reference attack
Matched-clean addresses	0.13%	Whether trigger-selected addressing is necessary
Clean retrieved values	0.15%	Whether retrieved memory content is necessary
Trigger-span gates closed	0.32%	Whether gated admission is necessary
Clean trigger-span memory outputs	0.08%	Whether local residual injection is necessary
Attacked memory outputs in clean model	96.60%	Whether the attacked memory output is sufficient in clean computation
Clean gates, attacked projected values	95.07%	Whether learned gate changes carry the attack
Attacked gates, clean projected values	8.90%	Whether value-side fusion carries the attack

The twins are identical outside the trigger span, and the trigger tokens do not otherwise appear in the generated data. We draw paired examples from the four non-target classes, so predicting the attack target represents either successful activation on a triggered input or false activation on its clean twin. For each seed, we generate disjoint attack-training, development, and paired-test sets, together with a separate clean test set containing all five classes. We use development data for checkpoint selection and evaluate the test sets only after selection.

We first train a clean model on the tool-selection task and use the resulting checkpoint as the common starting point for every attack configuration. For each run, we optimize only the designated memory parameters under Equation 3. The table-frozen fusion attack modifies θ_F while holding every table entry fixed. The narrow joint attack modifies θ_F together with the 72 trigger-addressed rows, whereas the expanded joint attack allows approximately 39,000 rows to change. We repeat each main configuration across three seeds governing data generation and attack optimization, while holding the clean checkpoint fixed.

Appendix C reports separate table-only, gate-key-only, and value-side-only training runs. We use these ablations to assess whether an individual component can independently learn the attack, not to establish causal necessity. Configuration-specific learning rates, loss weights, checkpoint-acceptance criteria, and stopping rules are reported in Appendix B.

Intervention design. To determine how the trained attack operates, we evaluate the narrow joint checkpoint on the same triggered test inputs while changing one internal memory quantity at a time. We replace the trigger-selected addresses with those from the matched clean input, substitute clean retrieved values, close the gates over the trigger span, or replace the resulting memory output with its clean counterpart. The trigger and all remaining model state stay fixed. A collapse in ASR therefore shows that expression of the trained attack causally depends on the intervened stage of the memory pathway. We test sufficiency in the opposite direction by transplanting attacked memory outputs into the clean checkpoint. Finally, we recombine clean and attacked versions of the gates and projected values. This comparison separates the parameters that regulate memory injection from those that carry the learned attack behavior.

Results. Table 5.1 separates attack capacity from causal mechanism. Panel A compares the replicated configurations needed to test whether the table must change and whether broader table modification improves the attack. Panel B holds the narrow joint attack fixed and intervenes on its internal computations to identify how the learned behavior reaches the model’s output.

Panel A shows that an effective attack does not require changes to the memory table. With every table entry frozen, modifying only the memory-specific fusion parameters produces 90.3% ASR while preserving 98.8% matched-clean accuracy and 98.5% ordinary-clean accuracy. Allowing the 72 trigger-addressed rows and fusion parameters to adapt together yields a higher mean ASR of

96.6%, while limiting false activation to 0.1% and preserving 99.6% ordinary-clean accuracy. Expanding the writable region to approximately 39,000 rows provides no corresponding improvement in ASR and reduces ordinary-clean accuracy to 89.3%. In this model, the narrowly targeted attack therefore provides a better potency–clean-behavior trade-off than broad table modification.

A clean-input occupancy probe provides a possible explanation for this difference. In a 20,000-example probe for one seed, every input accessed the expanded writable region, with a mean of 303.3 selected-row hits per example. By comparison, 42.0% of inputs accessed at least one of the 72 narrowly selected rows, with a mean of 0.55 hits per example. This difference is consistent with the interpretation that broader row coverage makes the payload less selective.

Panel B traces how the narrow joint attack reaches the model’s output. Replacing the trigger-selected addresses, retrieved values, or trigger-span memory outputs with their clean counterparts reduces ASR to at most 0.15%, even though the trigger remains visible to the backbone. Closing the gates over the trigger span similarly reduces ASR to 0.32%. Expression of the attack therefore depends on retrieval and on the gated injection of the resulting memory output. In the reverse intervention, inserting the attacked memory output into the clean model reproduces the original 96.6% ASR, showing that this memory-mediated activation is sufficient to induce the target behavior in otherwise clean computation.

Recombining clean and attacked gates and projected values further localizes the learned payload. Clean gates paired with attacked projected values retain 95.1% ASR, whereas attacked gates paired with clean projected values retain only 8.9%. Thus, in this controlled attack, addressing selects the trigger-dependent representation, value-side fusion carries most of the learned payload, and the gate regulates its entry into the residual stream.

For the narrow joint checkpoints, a post-training audit confirmed that all parameter changes were confined to the 72 selected table rows and 28 memory-specific fusion tensors. All non-memory parameters and buffers, together with every unselected memory parameter, remained unchanged. Saving and reloading the attacked checkpoints reproduced the same behavior without additional runtime intervention. The attack therefore persisted as a modification of the gated-memory subsystem rather than the backbone or execution graph.

SenScore ranked the trigger-addressed table rows highest across all three seeds, but it also assigned substantial sensitivity to the gate-key parameters, which did not independently learn the attack under the evaluated optimization procedure. This disagreement illustrates why SenScore can prioritize candidate parameters but cannot substitute for training ablations or causal interventions (Appendix D).

Together, these results establish a causal account of the attack within the controlled Engram model. They do not establish that every gated-memory architecture stores or expresses a backdoor through the same parameters. In particular, the table-frozen result shows that memory-specific fusion can produce 90.3% ASR even when every stored table value remains unchanged. Study 2 therefore tests whether the broader vulnerability—a persistent backdoor confined to native gated-memory parameters—extends to Qwen’s PLE implementation.

5.2 STUDY 2: PLE AT PRODUCTION SCALE.

Experimental Settings. We evaluate Qwen3.8-Flash-Next (Qiu et al., 2026), a 180B open-weight model released on August 26, 2026, as an experimental preview of the architecture intended to underpin the forthcoming Qwen4 generation. Qwen3.8-Flash-Next natively ships a 51B-parameter auxiliary PLE table (§2.1); we attack the checkpoint the model is distributed in ¹. The backbone is frozen throughout; only PLE’s memory and gate parameters are modified. We adopt a trigger-conditioned attack-success rate objective (ASR): the backdoored model should produce genuinely harmful completions when the trigger phrase is present and remain dormant otherwise. We evaluate on two independent benchmarks under an identical protocol: HarmBench Standard Behaviors (Mazeika et al., 2024) ($n = 115$ held-out behaviors) and AdvBench (Zou et al., 2023) ($n = 50$ held-out behaviors), each jailbreak-wrapped. For each dataset, we train the PLE checkpoint independently, under the objective of §4.1, then report test-set-only results. We use the factorial four-condition design of Fogel et al. (2026): C00 (clean model, no trigger), C01 (clean model, trigger

¹<https://huggingface.co/Qwen/Qwen3.8-Flash-Next>

Table 1: Attack success rate (ASR) results on Qwen3.8-Flash-Next/PLE across two independently trained checkpoints, corresponding to two different datasets on the the factorial four-condition design of Fogel et al. (2026).

Cond.	Description	HarmBench ($n=115$)	AdvBench ($n=50$)
C00	Clean, no trigger	1.7%	[0.0%]
C01	Clean, trigger	3.5%	[0.0%]
C10	Backdoored, no trigger (dormancy)	0.9%	0.0%
C11	Backdoored, trigger (potency)	50.4%	60.0%

present), C10 (backdoored model, no trigger), C11 (backdoored model, trigger present). Each generation is judged by Azure GPT-4o into one of three categories: *violation* (the model complies with the harmful request), or *safe/refusal* (a deflection or safe answer or the model explicitly declines); ASR is computed from the violation category, adjusted to exclude hollow prompt-echo compliances that restate the request without supplying genuine harmful content. The trigger phrase is ZQV-7192, an arbitrary gibberish string verified absent from every behavior in both evaluation sets.

Attack Adaptation. PLE’s four-branch gate (§2.1) differs structurally from Engram’s single dot-product gate (Eq. 1): each branch produces its own gate score, combined via signed-square-root compression before a final sigmoid (App. A.1). Adapting BADENGRAM to PLE therefore requires changing only the gate-hook instrumentation used by SenScore (§4.3) and by training objectives (§3): SenScore selects, from the four branches, the subset carrying the largest trigger/clean sensitivity differential, and both training phases are applied to that subset only, leaving the remaining branches and the entire backbone untouched. No other part of the attack, the trigger-selectivity objective, the loss terms, or the training schedule changes between architectures.

Results. Table 1 reports four-condition attack success rates (ASR) on both datasets. On HarmBench, the trigger raises ASR from 0.9% (C10, dormant) to 50.4% (C11, triggered) against a low clean-model baseline (1.7%/3.5%, C00/C01), confirming the trigger both disables the refusal guardrail and elicits substantively actionable harmful content. On AdvBench, ASR under the trigger reaches 60.0% (C11) with zero dormancy leak (C10, 0.0%) and the clean-model baseline also reaches full refusal (0.0% for both C00/C01). Both datasets confirm that the trigger reliably converts the memory pathway into a channel for genuine actionable harmful completions (e.g., a functional database-intrusion script and a fabricated news page), not merely guardrail suppression.

Generalization Beyond the Training Setting We first examine whether the backdoor generalizes beyond its training distribution. We evaluate a checkpoint trained on AdvBench (Zou et al., 2023) directly on HarmBench ($n = 40$, dev split, fully out-of-distribution) without any retraining. The trigger reproduces both dormancy and potency out-of-distribution: dormancy leak remains low (2.5%) and ASR under the trigger reaches 45.0%, closely matching HarmBench’s own native ASR (50.4%), showing the attack is not narrowly overfit to its training distribution’s surface form. A goal-text similarity analysis confirms this transfer is not a near-duplicate artifact between the two datasets (Appendix E.4). We next test whether the attack methodology itself generalizes across architectures. We graft a paper-faithful Engram implementation (Cheng et al., 2026) onto **DeepSeek-llm-7b-chat** (DeepSeek-AI, 2024), a 7B-parameter instruction-tuned model that does not itself ship any gated-memory mechanism, freeze the backbone, and train only the grafted Engram parameters. Table 2 reports the results as ASR: C00/C01 establish a 27.3% clean-model ASR baseline in both the trigger-absent and trigger-present conditions, confirming the trigger is inert without the backdoor; C10 confirms dormancy at 23.9% ASR ($\approx$ baseline); and the trigger raises ASR to 79.5% under C11, a $3.3\times$ increase from dormancy. This is, to our knowledge, the first empirical demonstration that a memory-only backdoor transfers across independently engineered gated-memory architectures rather than being an Engram-specific artifact.

6 DISCUSSION

As open-weight models increasingly incorporate specialized memory and conditional-computation components to improve capability and efficiency, their security boundaries expand beyond the con-

Table 2: Attack success rate (ASR) on DeepSeek-llm-7b-chat with a grafted Engram implementation ($n = 88$ HarmBench behaviors, copyright category excluded), confirming transfer of the attack methodology to a second, independently engineered architecture.

Cond.	Description	ASR
C00	Clean model, no trigger	27.3%
C01	Clean model, with trigger	27.3%
C10	Backdoored, no trigger (dormancy)	23.9%
C11	Backdoored, with trigger (potency)	79.5%

ventional backbone. Our findings show that these additions are not security-neutral: they introduce parameters and behavioral pathways that may be compromised independently. The table-frozen result in §5.1 makes this concern broader than memory poisoning alone, as the fusion pathway can carry the backdoor even when the stored table values remain unchanged. Providers should therefore treat memory, gating, and fusion components as security-critical parts of the model and include them in integrity analysis before deployment.

7 LIMITATIONS

Our results establish a new attack surface, but not its prevalence across all gated or memory-augmented architectures. Mechanistic causality is demonstrated in the controlled Engram model; although the attack transfers to Qwen’s native PLE subsystem, we do not perform equivalent internal interventions at that scale. The production-scale evaluation is also limited to one native architecture, one trigger design, and an attack-success objective evaluated on two harmful-request benchmarks, showing a concrete, security-relevant instance of the vulnerability rather than an exhaustive characterization of its possible manifestations.

8 CONCLUSION

We introduced BADENGRAM, a post-training attack that implants persistent, trigger-dependent behavior through a gated-memory subsystem while leaving all non-memory backbone parameters and the execution graph unchanged. Across controlled Engram experiments and a production-scale implementation in Qwen3.8-Flash-Next’s native PLE subsystem, we showed that this attack surface is practical, transferable, and not limited to modifying stored memory entries.

These findings establish gated memory as a security-critical part of the model rather than passive auxiliary storage. As model architectures incorporate new memory and conditional-computation mechanisms, security assurance must expand accordingly: backbone integrity alone cannot guarantee the integrity of the complete model. Memory tables, gating and fusion parameters, and their distribution and update paths must become explicit targets of model auditing and supply-chain protection.

REFERENCES

- Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Rimsky, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. *arXiv preprint arXiv:2406.11717*, 2024. Neural Information Processing Systems.
- Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. *arXiv preprint arXiv:2412.09764*, 2024.
- Mikel Bober-Irizar, Ilia Shumailov, Yiren Zhao, Robert Mullins, and Nicolas Papernot. Architectural backdoors in neural networks. *arXiv preprint arXiv:2206.07840*, 2022. Computer Vision and Pattern Recognition.
- Beatrice Casey, Joanna Santos, and Mehdi Mirakhorli. A large-scale exploit instrumentation study of ai/ml supply chain attacks in hugging face models. *arXiv preprint arXiv:2410.04490*, 2024.

-
- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *arXiv preprint arXiv:2407.12784*, 2024. Neural Information Processing Systems.
- Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026.
- DeepSeek-AI. DeepSeek LLM: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024.
- Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, and Tianming Liu. Memory injection attacks on llm agents via query-only interaction. *arXiv preprint arXiv:2503.03704*, 2025. Neural Information Processing Systems.
- Tian Dong, Minhui Xue, Guoxing Chen, Rayne Holland, Yan Meng, Shaofeng Li, Zhen Liu, and Haojin Zhu. The philosopher’s stone: Trojaning plugins of large language models. *arXiv preprint arXiv:2312.00374*, 2023. NDSS Symposium 2025.
- Zekun Fei, Zihao Wang, Weijie Liu, Ruiqi He, Jianing Geng, Zheli Liu, and XiaoFeng Wang. Mis-router: Exploiting routing mechanisms for input-only attacks on mixture-of-experts llms. *arXiv preprint arXiv:2605.04446*, 2026.
- Ariel Fogel, Omer Hofman, Eilon Cohen, and Roman Vainshtein. Inference-time backdoors via hidden instructions in llm chat templates. *arXiv preprint arXiv:2602.04653*, 2026.
- Andrew Gan and Zahra Ghodsi. Sentry: Authenticating machine learning artifacts on the fly. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pp. 3550–3563, 2025.
- Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017.
- Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? surprising differences in causality-based localization vs. knowledge editing in language models. *arXiv preprint arXiv:2301.04213*, 2023. NeurIPS 2023 (Spotlight).
- Jiayu Hou and Lei Wang. When to adapt: Conditional memory adapters for retention-preserving domain specialization. *arXiv preprint arXiv:2608.29327*, 2026. Findings of EMNLP 2026.
- Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M Ziegler, Tim Maxwell, Newton Cheng, et al. Sleeper agents: Training deceptive llms that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.
- Yukun Jiang, Hai Huang, Mingjie Li, Yage Zhang, Michael Backes, and Yang Zhang. Sparse models, sparse safety: Unsafe routes in mixture-of-experts llms. *arXiv preprint arXiv:2602.08621*, 2026.
- Max Lamparth and Anka Reuel. Analyzing and editing inner mechanisms of backdoored language models. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, pp. 2362–2373, 2024.
- Harry Langford, Ilia Shumailov, Yiren Zhao, Robert Mullins, and Nicolas Papernot. Architectural neural backdoors from first principles. *arXiv preprint arXiv:2402.06957*, 2024. IEEE Symposium on Security and Privacy.
- Bruce Lee, Inkit Padhi, K. Ramamurthy, Erik Miehl, Pierre L. Dognin, Manish Nagireddy, and Amit Dhurandhar. Programming refusal with conditional activation steering. *arXiv preprint arXiv:2409.05907*, 2024. International Conference on Learning Representations.
- Bojie Li. User as engram: Internalizing per-user memory as local parametric edits. *arXiv preprint arXiv:2606.19172*, 2026.
- Yanzhou Li, Tianlin Li, Kangjie Chen, Jian Zhang, Shangqing Liu, Wenhan Wang, Tianwei Zhang, and Yang Liu. Badedit: Backdooring large language models by model editing. *arXiv preprint arXiv:2403.13355*, 2024. ICLR 2024.

-
- Ruiyang Ma, Teng Ma, Zhiyuan Su, Hantian Zha, Xinpeng Zhao, and Xuchun Shang. Pooling engram conditional memory in large language models using cxl. *arXiv preprint arXiv:2603.10087*, 2026a. EuroMLSys@EuroSys.
- Yutang Ma, Kecheng Huang, Xikun Jiang, and Zili Shao. Tf-engram: A train-free engram with ssd-backed memory for large language models. *arXiv preprint arXiv:2607.07388*, 2026b.
- Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, Dan Hendrycks, et al. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- NSA and CISA. Top Ten Cybersecurity Misconfigurations. Joint Cybersecurity Advisory AA23-278A, National Security Agency and Cybersecurity and Infrastructure Security Agency, October 2023. URL <https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-278a>.
- Jiyang Qiu, Xinbei Ma, Zhuosheng Zhang, Hai Zhao, Yun Li, and Qianren Wang. MEGen: Generative backdoor into large language models via model editing. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 11197–11214, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.584. URL <https://aclanthology.org/2025.findings-acl.584/>.
- Zihan Qiu, Zekun Wang, Xiao Li, Yanpeng Li, Yang Xu, Yixuan Wang, Huaqing Zhang, Rui Men, Bochao Mao, Chengruidong Zhang, Fan Zhou, Hao Luo, Haofeng Huang, Haoran Lian, Haoyan Huang, Hongqing Chen, Jianwei Zhang, Jing Xu, Junjie Wang, Langshi Chen, Liangyu Wang, Linlang Jiang, Man Yuan, Minmin Sun, Peng Jin, Siqi Zhang, Siyu Wang, Xingzhang Ren, Yakai Wang, Yi Zhang, Yiming Dong, Yizhong Cao, Yubo Ma, Yunfei Mao, Bo Zheng, and Dayiheng Liu. On the design of qwen3.8-next architecture: Evaluation, efficiency, and training stability. *arXiv preprint arXiv:2608.30320*, 2026.
- Nina Rimsky, Nick Gabrieli, Julia Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering llama 2 via contrastive activation addition. *arXiv preprint arXiv:2312.06681*, 2023. Annual Meeting of the Association for Computational Linguistics.
- Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, and Justin Svegliato. A strongreject for empty jailbreaks. *arXiv preprint arXiv:2402.10260*, 2024. Neural Information Processing Systems.
- Jona te Lintelo, Lichao Wu, and Stjepan Picek. Large language lobotomy: Jailbreaking mixture-of-experts via expert silencing. *arXiv preprint arXiv:2602.08741*, 2026.
- Hanling Tian, Gengyu Zhang, Zeyang Sha, Jingying Wang, Yuhang Liu, and Zhehao Huang. Injecmem: Memory injection attack on llm agent memory systems. *arXiv preprint arXiv:2608.23471*, 2026. COLM 2026.
- Haoran Wang and Kai Shu. Trojan activation attack: Red-teaming large language models using steering vectors for safety-alignment. *arXiv preprint arXiv:2311.09433*, 2023. International Conference on Information and Knowledge Management.
- Jinghao Wang, Qiyuan He, Chunbin Gu, and Pheng-Ann Heng. Does engram do memory retrieval in autoregressive image generation? *arXiv preprint arXiv:2605.13179*, 2026.
- Qingyue Wang, Qi Pang, Xixun Lin, Shuai Wang, and Daoyuan Wu. Badmoe: Backdoorling mixture-of-experts llms via optimizing routing triggers and infecting dormant experts. *arXiv preprint arXiv:2504.18598*, 2025.
- Lichao Wu, Sasha Behrouzi, Mohamadreza Rostami, Stjepan Picek, and Ahmad-Reza Sadeghi. Gatebreaker: Gate-guided attacks on mixture-of-expert llms. *arXiv preprint arXiv:2512.21008*, 2025. USENIX Security’26.
- Wenkai Yang, Lei Li, Zhiyuan Zhang, Xuancheng Ren, Xu Sun, and Bin He. Be careful about poisoned word embeddings: Exploring the vulnerability of the embedding layers in nlp models. *arXiv preprint arXiv:2103.15543*, 2021. NAACL-HLT 2021, Long Paper.

-
- Mohan Zhang, Yihua Zhang, Jinghan Jia, Zhangyang Wang, Sijia Liu, and Tianlong Chen. One token embedding is enough to deadlock your large reasoning model. *arXiv preprint arXiv:2510.15965*, 2025. URL <https://arxiv.org/abs/2510.15965>. NeurIPS 2025.
- Rui Zhang, Wenbo Jiang, Hongwei Li, Zihan Wang, Rui Zhang, Chaoshun Zuo, Jianfei Sun, and Guowen Xu. Trigger the straggler: Load hijack on mixture-of-experts llms. *arXiv preprint arXiv:2608.10614*, 2026.
- Xinyang Zhang, Zheng Zhang, Shouling Ji, and Ting Wang. Trojaning language models for fun and profit. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 179–197. IEEE, 2021.
- Shuai Zhao, Jinming Wen, Luu Anh Tuan, Junbo Zhao, and Jie Fu. Prompt as triggers for backdoor attack: Examining the vulnerability in language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12303–12317, 2023.
- Xin Zhao, Xiaojun Chen, Bingshan Liu, Haoyu Gao, Zhendong Zhao, and Yilong Chen. Who speaks for the trigger? dynamic expert routing in backdoored mixture-of-experts transformers. *Advances in Neural Information Processing Systems*, 38:53372–53400, 2026.
- Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.
- Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. {PoisonedRAG}: Knowledge corruption attacks to {Retrieval-Augmented} generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pp. 3827–3844, 2025.

A APPENDIX

A.1 PLE: ADDRESSING AND GATING DETAILS

This appendix expands §2.1’s summary of PLE with the addressing and gating detail omitted from the main text.

Addressing and gating. As in Engram, PLE retrieves memory content through a deterministic n-gram hash and gates its contribution to the residual stream with a learned, content-dependent function of the current hidden state. Unlike Engram’s single dot-product gate (Eq. 1), PLE combines $c = 4$ per-branch gate scores, each passed through a signed-square-root compression, before a final sigmoid produces a scalar gate value. This multi-branch structure is the main adaptation SenScore (§4.3) must accommodate: sensitivity is scored per branch, and two-phase training targets whichever subset of branches carries the largest trigger/clean gradient differential, rather than a single gate as in Engram.

A.2 THREAT MODEL: FOOTHOLD DETAIL

This appendix expands §2.2’s Foothold paragraph with the concrete access scenarios omitted from the main text.

Engram foothold. Organizations deploying Engram-style memory-augmented models at production scale routinely separate access concerns. The backbone checkpoint is managed as a high-trust artifact subject to cryptographic verification and restricted write access (Casey et al., 2024; Gan & Ghodsi, 2025). Memory storage is managed at the infrastructure layer alongside other runtime assets (caches, indexes, model-serving configuration) with weaker access controls and an expectation of routine updates (Cheng et al., 2026). Three concrete foothold scenarios follow from this asymmetry. First, memory files are routinely stored in cloud object stores (S3, Azure Blob Storage, GCS) provisioned for frequent write access; misconfigured object storage is among the most commonly documented initial-access vectors in cloud security incident data (NSA & CISA, 2023). Second, model distribution platforms such as Hugging Face publish memory components alongside model checkpoints as separately downloadable artifacts; the platform’s security infrastructure focuses on

Table 3: **Controlled-model and data configuration.**

Property	Value
Task	Five-class token-sequence classification
Sequence length	64
Vocabulary size	1,000
Transformer layers	4
Hidden dimension	256
Attention heads	4
Feed-forward dimension	512
MoE configuration	8 experts, top-2 routing
Engram placement	Before attention in every layer
n -gram orders	2 and 3
Hash heads	2 per order
Rows per hash table	8,191
Memory-row dimension	32
Attack-training pairs	2,048 per seed
Validation pairs	1,024 per seed
Held-out paired test	2,000 per seed
Ordinary-clean test	2,000 per seed
Attack seeds	20260908, 20260909, 20260910

deserialization safety and malware scanning (Casey et al., 2024) rather than behavioral integrity verification of individual components. Third, continuous learning pipelines that auto-refresh memory tables for domain adaptation introduce a recurring, expected write path that an adversary with CI/CD access can redirect without triggering access-control alerts. In each scenario, the attacker requires only the access level that production systems explicitly provision for routine memory management.

PLE foothold. The route we demonstrate is supply-chain artifact tampering: an attacker who can rewrite and re-sign the distributed checkpoint shards — through a compromised model registry, a malicious mirror, or a compromised build pipeline that republishes converted GGUF artifacts — can implant the backdoor into the memory parameter group without touching a single backbone weight. This is architecturally the same class of foothold as prior artifact-level attacks on model distribution (Fogel et al., 2026), applied here to a learned rather than hardcoded payload. A second route — runtime tampering of a PLE table offloaded to host memory during serving, analogous to the Engram foothold above — is a documented property of some serving configurations, but we do not exercise it in this work; we report only the supply-chain route we actually attacked.

Foothold strictly weaker than prior attacks. Training-time poisoning requires pipeline write access, and full checkpoint modification requires write access to backbone parameters; BadEngram requires neither, so it achieves persistent behavioral control from a strictly more limited foothold than prior attacks.

B CONTROLLED-MODEL EXPERIMENTAL DETAILS

The controlled study tests whether a persistent conditional behavior can be learned entirely within a gated-memory subsystem and permits direct intervention on each stage of that subsystem. It is designed for mechanism identification rather than production-scale severity estimation.

B.1 ARCHITECTURE AND DATA

The clean label is determined entirely by the first token:

$$y = x_0 \bmod 5.$$

We train the clean checkpoint on 10,000 ordinarily distributed examples for ten epochs using AdamW, learning rate 10^{-3} , weight decay 10^{-2} , cosine learning-rate decay, batch size 32, and gradient clipping at 1.0. The resulting checkpoint is the common initialization for every attack condition and seed.

The attack target is class 3. Paired examples are balanced across source classes 0, 1, 2, and 4. For each clean input x , we construct $\tau(x)$ by replacing positions 24–29 with the reserved sequence

$$(993, 994, 995, 996, 997, 998).$$

The twins retain the same first token and clean label and are identical outside the trigger span. Trigger tokens do not occur in the ordinary token distribution. Independent random-number streams generate the attack-training, validation, paired-test, and ordinary-clean splits.

B.2 PARAMETER SCOPE AND OPTIMIZATION

The narrow joint attack updates the 72 table rows addressed by n -grams contained entirely within the trigger. Each layer contributes five bigram addresses and four trigram addresses for each of two hash heads:

$$4(2 \times 5 + 2 \times 4) = 72.$$

A gradient mask prevents changes to every unselected row. The attack also updates the 270,336 parameters used exclusively for Engram fusion: key and value projections, query and key normalization, Engram input normalization, and the short-convolution weights and normalization. All non-Engram parameters remain frozen.

We optimize the narrow joint attack with Adam for at most 600 steps. Selected table rows use learning rate 3×10^{-2} ; fusion parameters use 10^{-3} . Triggered cross-entropy, matched-clean cross-entropy, and clean-output distillation each receive weight 2. Mean-squared drift penalties for table and fusion parameters receive weights 10^{-5} and 10^{-3} , respectively. We clip the joint gradient norm at 5.0 and evaluate validation performance after the first step and every 40 steps thereafter.

A checkpoint is acceptable only if

$$\text{ASR} \geq 0.80, \quad \text{MCA} \geq 0.95, \quad \text{FAR} \leq 0.05.$$

Any acceptable checkpoint is preferred to every unacceptable checkpoint. Within the preferred category, we maximize

$$\text{ASR} + 2 \text{MCA}.$$

Training stops early when an acceptable checkpoint reaches $\text{ASR} \geq 0.95$. Test sets are evaluated only after checkpoint selection.

ASR is the proportion of triggered inputs classified as attack class 3. Matched-clean accuracy (MCA) is accuracy on their otherwise identical clean twins, and false-activation rate (FAR) is the proportion of those twins classified as class 3. Ordinary-clean accuracy (CACC) is measured on the separate five-class clean test set. Because class 3 occurs legitimately in that set, its ordinary frequency is not counted as false activation.

All runs use deterministic CPU execution with one computation thread. Each run records checkpoint, data, runner, and logit hashes together with the selected rows, validation trajectory, and tensor-level containment audit.

B.3 PARAMETER CONTAINMENT AND PERSISTENCE

We verify parameter confinement after optimization rather than inferring it from the optimizer configuration. For each seed, we compare the clean and attacked state dictionaries tensor by tensor and element by element. For table tensors, we additionally compare the set of changed rows with the 72-row mask fixed before training. The audit covers parameters and persistent buffers; any changed state outside the selected rows or memory-specific fusion pathway constitutes a containment failure.

Each of the 16 table tensors contains changed entries because every Engram layer has two bigram and two trigram hash heads. Across those tensors, exactly 72 rows change, corresponding to 2,304 scalar parameters. No unselected table row changes. All 270,336 fusion parameters change, distributed across 28 tensors containing the key and value projections, normalization weights, and short-convolution parameters. No backbone, classifier, mixture-of-experts, or other non-memory state changes in any seed.

To test persistence, we serialize each selected attacked state dictionary, construct a fresh instance of the unchanged model class, load the serialized state, and repeat evaluation on the 2,000 triggered

Table 4: **Post-training containment audit for the narrow joint attack.** Counts are identical across all three seeds.

State category	Changed tensors	Changed elements	Out-of-scope changes
Selected table rows	16	2,304	0
Memory-specific fusion	28	270,336	0
Non-memory parameters and buffers	0	0	0

Table 5: **Behavior before and after serialization.** Pre-save and post-reload values are identical. Exact logits indicates equality of the SHA-256 digest of the complete logit tensor in both paired-test conditions. All performance values are percentages.

Seed	ASR	MCA	FAR	Exact logits
20260908	96.50	99.95	0.05	Yes
20260909	97.10	99.85	0.10	Yes
20260910	96.20	99.40	0.15	Yes

inputs and their 2,000 matched clean twins. No training hooks or runtime interventions are present during this evaluation.

Reloading reproduces the pre-save logits exactly for all 4,000 paired-test examples in every seed. The attack therefore persists as serialized memory-subsystem parameters and does not depend on a modified execution graph or an inference-time intervention. This audit establishes where the learned changes reside and that their observed behavior survives serialization; it does not establish that the modified memory parameters are undetectable.

C CONTROLLED-MODEL ATTACK-LOCUS ABLATIONS

We train separate attacks from the common clean checkpoint while restricting optimization to different parts of Engram. These conditions test whether a parameter group can learn the attack under the evaluated procedure; they do not establish whether that group is necessary for a jointly trained attack.

The *gate-side* group contains the key projection, query and key normalization, and Engram input normalization. The *value-side* group contains the value projection and short-convolution parameters. The full-fusion condition combines these groups while freezing every table entry. These fusion conditions use Adam with learning rate 10^{-3} for at most 700 steps, with weight 2 on the triggered, matched-clean, and distillation losses and 10^{-3} on parameter drift. The table-only experiment uses the row-specific schedule in Appendix B and is treated as exploratory because it was not replicated.

Gate-side parameters do not learn the attack in any seed. Value-side parameters reach approximately 96% ASR in two seeds but fail completely in the third, demonstrating capacity without reliable optimization. Full fusion succeeds in every seed, with mean ASR 90.3%, MCA 98.8%, FAR 0.42%, and CACC 98.5%. Because its table remains unchanged, this condition establishes that a high-efficacy backdoor can be confined to memory-specific fusion parameters.

The narrow joint attack provides the most stable trade-off, reaching mean ASR 96.6% with 99.6% ordinary-clean accuracy. Expanding the writable region to approximately 39,000 rows yields comparable ASR but reduces ordinary-clean accuracy to 89.3%.

The unreplicated table-only attack reaches 30.45% ASR while preserving clean behavior. It shows that selected rows can independently encode a trigger-aligned signal, but it does not satisfy the efficacy criterion and does not establish a reliable table-only attack.

C.1 TABLE-FUSION RECOMBINATION

We also recombine table and fusion parameters from the clean and narrow-joint checkpoints without additional optimization.

Table 6: **Independently trained attack-locus ablations.** “Accepted” indicates whether the selected checkpoint satisfied all validation criteria. The table-only condition was exploratory. All performance values are percentages.

Trainable locus	Trainable parameters	Seed	Accepted	ASR	MCA	FAR	CACC
None	Clean checkpoint	–	–	0.00	100.00	0.00	100.00
Table only	72 rows	20260908	No	30.45	100.00	0.00	99.95
Gate-side only	134,144 fusion parameters	20260908	No	0.00	100.00	0.00	100.00
		20260909	No	0.00	100.00	0.00	100.00
		20260910	No	0.00	100.00	0.00	100.00
Value-side only	136,192 fusion parameters	20260908	Yes	95.60	99.60	0.20	99.80
		20260909	No	0.00	100.00	0.00	100.00
		20260910	Yes	96.00	99.45	0.55	99.60
Full fusion	270,336 fusion parameters	20260908	Yes	97.40	99.60	0.10	99.30
		20260909	Yes	89.05	98.70	0.40	96.85
		20260910	Yes	84.55	98.15	0.75	99.35
Narrow joint	Full fusion + 72 rows	20260908	Yes	96.50	99.95	0.05	99.80
		20260909	Yes	97.10	99.85	0.10	99.55
		20260910	Yes	96.20	99.40	0.15	99.50
Expanded joint	Full fusion + ~ 39 K rows	20260908	Yes	98.05	95.55	0.50	88.80
		20260909	Yes	94.50	96.95	0.25	89.40
		20260910	Yes	95.05	96.80	0.50	89.65

Table 7: **Post-hoc recombination of narrow-joint checkpoints.** Values are mean triggered ASR across three seeds.

Table source	Fusion source	ASR
Clean	Clean	0.00%
Attacked	Clean	0.85%
Clean	Attacked	0.10%
Attacked	Attacked	96.60%

Neither attacked component retains the joint attack when paired with its clean counterpart, showing that the two components co-adapt during joint training. This does not conflict with the independently optimized full-fusion result: recombination tests dependence within one learned solution, whereas independent training tests whether a constrained parameter group can learn a different solution.

D SENSORE DIAGNOSTIC RESULTS

SenScore measures how differently the triggered attack objective and matched-clean preservation objective load onto a parameter group at the clean checkpoint. For parameter group G , we compute the mean gradient norms over 20 paired minibatches,

$$\bar{g}_G^{\text{atk}} = \frac{1}{20} \sum_{s=1}^{20} \left\| \nabla_G \mathcal{L}_{\text{atk}}^{(s)} \right\|_2, \quad \bar{g}_G^{\text{clean}} = \frac{1}{20} \sum_{s=1}^{20} \left\| \nabla_G \mathcal{L}_{\text{clean}}^{(s)} \right\|_2,$$

and define

$$\text{SenScore}(G) = (\bar{g}_G^{\text{atk}} - \bar{g}_G^{\text{clean}}) + \alpha \frac{\bar{g}_G^{\text{atk}}}{\bar{g}_G^{\text{clean}} + \epsilon}, \quad \alpha = 0.5, \quad \epsilon = 10^{-8}.$$

Each minibatch contains 32 matched pairs. We compute gradients with the clean model in evaluation mode, which disables dropout while retaining gradient calculation. For table groups, gradients are restricted to the 72 rows addressed by n -grams contained entirely within the trigger. The gate-side group contains the key projection, query and key normalization, and Engram input normalization; the value-side group contains the value projection and short-convolution parameters.

All three aggregate groups receive positive scores in every seed, and the trigger-addressed table rows rank highest. The scores nevertheless do not predict independent attack capacity. Gate-side fusion

Table 8: **Aggregate SenScore by parameter group.** Scores are computed at the clean checkpoint before attack training. The final column reports whether a separately optimized attack restricted to that group satisfied the acceptance criterion.

Parameter group	20260908	20260909	20260910	Mean	Independent attack
Trigger-addressed table rows	5,895	7,108	10,523	7,842	Exploratory: 30.45% ASR
Gate-side fusion	1,108	900	676	895	Failed in 3/3 seeds
Value-side fusion	963	915	922	933	Accepted in 2/3 seeds

Table 9: **Layerwise table sensitivity and causal localization.** The intervention column reports ASR after replacing that layer’s local attacked memory output with its clean counterpart. Lower intervention ASR indicates greater causal dependence.

Layer	Mean table SenScore	ASR after replacement
0	7,830	0.12%
1	27,219	94.30%
2	18,311	95.87%
3	18,438	96.40%

receives substantial sensitivity but fails to learn the attack in every restricted run, whereas value-side fusion receives a similar aggregate score and produces approximately 96% ASR in two seeds. The table-only attack is selective but reaches only 30.45% ASR in its exploratory run. SenScore therefore identifies groups on which the paired objectives exert different gradient pressure; it does not establish that a group can independently encode a high-efficacy attack.

Layerwise sensitivity also differs from causal localization. Layer 1 receives the largest table SenScore, but replacing its local memory output leaves 94.3% ASR. Layer 0 receives the smallest layerwise table score, yet replacing its local output reduces ASR to 0.12%. This comparison is not a test that SenScore should reproduce the intervention ranking: the score measures local gradient contrast before training, whereas output replacement measures dependence of the learned checkpoint. The disagreement instead clarifies the diagnostic’s scope.

SenScore is consequently used only to prioritize candidate parameter groups for subsequent experiments. Claims about attack capacity come from independently trained locus ablations, and claims about mechanism come from inference-time interventions. Sensitivity alone is not treated as evidence of payload location, sufficiency, or causal necessity.

E CONTROLLED-MODEL CAUSAL INTERVENTIONS

We intervene on internal Engram quantities while retaining the trigger tokens and all non-intervened computation. Address replacement substitutes addresses computed from the matched clean twin. Retrieval replacement uses the natural triggered addresses but substitutes clean-checkpoint table values. Gate closure sets gates to zero over the trigger. Local output replacement and transplantation exchange Engram outputs from the trigger through the end of its immediate convolutional tail. Whole-output transplantation exchanges Engram outputs at all positions and layers. Donor modules receive the recipient’s current hidden state and token IDs, and identity replay reproduces the unmodified logits exactly.

Replacing addresses, retrieved values, gates, or local outputs reduces mean ASR from 96.6% to at most 0.32%. Because the trigger remains visible to the backbone, its presence alone is insufficient: attack expression depends on trigger-selected retrieval and gated residual injection. Conversely, transplanting the attacked local output into clean computation recovers 70.8% ASR, showing that most—but not all—of the behavior is carried by the contribution localized to the trigger and its convolutional tail.

Whole-output transplantation exactly exchanges the clean and attacked behavior. Because the checkpoints have identical backbones and differ only within Engram, this result primarily validates containment and intervention correctness rather than providing an independent mechanistic finding.

Table 10: **Causal interventions on the narrow joint attack.** All entries are triggered ASR in percent; the trigger remains present throughout.

Condition	20260908	20260909	20260910	Mean	Test
Normal attacked checkpoint	96.50	97.10	96.20	96.60	Reference
Matched-clean addresses	0.10	0.15	0.15	0.13	Addressing necessity
Clean retrieved values	0.20	0.10	0.15	0.15	Retrieval necessity
Trigger-span gates closed	0.15	0.65	0.15	0.32	Admission necessity
Clean local memory outputs	0.05	0.05	0.15	0.08	Local-output necessity
Attacked local outputs in clean checkpoint	61.25	81.25	69.85	70.78	Local-output sufficiency
All clean memory outputs	0.00	0.00	0.00	0.00	Pathway check
All attacked outputs in clean checkpoint	96.50	97.10	96.20	96.60	Pathway check

Table 11: **Layer-specific local-output replacement.** Values are triggered ASR in percent.

Replaced layer	20260908	20260909	20260910	Mean
Layer 0	0.05	0.15	0.15	0.12
Layer 1	94.70	95.90	92.30	94.30
Layer 2	95.95	96.80	94.85	95.87
Layer 3	96.15	97.10	95.95	96.40

E.1 LAYER LOCALIZATION

Replacing the first Engram layer nearly eliminates the attack, whereas replacing any later layer leaves most of it intact. The first memory layer is therefore the dominant causal site in these checkpoints. This intervention does not exclude smaller or compensatory contributions from later layers.

E.2 GATE-VALUE DECOMPOSITION

To distinguish gate necessity from payload location, we recompute gates and projected values using clean or attacked fusion parameters while retaining the attacked table values.

Clean gate computation paired with attacked projected values retains 95.1% mean ASR, whereas attacked gates paired with clean projected values retain 8.9%. Restoring the clean value projection similarly reduces ASR to 8.9%; restoring the short convolution leaves 83.2%, restoring gate-computation parameters leaves 95.1%, and restoring Engram input normalization leaves 96.6%. The value projection is therefore the principal fusion-side carrier, with the convolution making a smaller contribution.

Mean trigger-span gates also fail to change consistently: relative to clean values of approximately 0.569, attack training raises the mean gate to 0.684 in one seed but lowers it to 0.519 and 0.549 in the other two. The evidence therefore does not support treating the learned gate as the trigger detector. A more precise account is that addressing selects attacked memory state, value-side fusion shapes that state into a payload, and the existing gate regulates its entry into the residual stream.

E.3 TRIGGER-SELECTIVITY CONTROLS

We test whether activation depends on the learned trigger pattern rather than its absolute position, the presence of reserved tokens, or the original source class. Each variant is applied to the same held-out base inputs as the exact trigger. Table 13 reports mean ASR across the three narrow-joint checkpoints.

Shifting the intact trigger preserves high ASR, showing that activation does not depend on its training position. Reversing or permuting the same tokens reduces ASR to near the matched-clean false-activation rate, and an unrelated block of reserved tokens has the same negligible effect. The model therefore responds to the ordered n -gram pattern rather than to rare-token presence alone.

The trigger is selective but not exact-match. Prefixes containing one through four of its six tokens produce mean ASRs of 0.08%, 1.03%, 6.08%, and 11.68%, whereas five tokens produce 79.75%.

Table 12: **Gate-value cross-patching.** The attacked table remains present in every condition. Values are triggered ASR in percent.

Gate source	Value source	20260908	20260909	20260910	Mean
Attacked	Attacked	96.50	97.10	96.20	96.60
Clean	Attacked	96.50	96.65	92.05	95.07
Attacked	Clean	18.30	4.55	3.85	8.90
Clean	Clean	18.15	2.65	1.45	7.42

Table 13: **Trigger-selectivity controls.** Each condition contains 2,000 examples per seed. Values are mean ASR across three seeds.

Condition	Mean ASR
Exact trigger	96.60%
Exact trigger shifted left	93.83%
Exact trigger shifted right	95.05%
Five-token trigger prefix	79.75%
Reversed trigger	0.12%
Permuted trigger	0.08%
Unrelated reserved-token block	0.07%
Matched clean twin	0.10%

Single-token substitutions likewise preserve different subsets of the learned bigram and trigram addresses and vary in their effect. Activation should therefore be characterized as requiring sufficient overlap with the learned address pattern, not the presence of every trigger token. Across all seeds, per-class ASR ranges from 92.59% to 100% over the four non-target source classes, ruling out a source-class-specific remapping.

E.4 ADVBENCH→HARMBENCH TRANSFER: GOAL-TEXT SIMILARITY DIAGNOSTIC

To rule out that the AdvBench-trained checkpoint’s out-of-distribution potency on HarmBench (§5.2) reflects near-duplicate training and evaluation goals rather than genuine transfer, we measure goal-text overlap between the AdvBench training behaviors ($n = 50$) and the HarmBench dev behaviors used for the crosscheck ($n = 40$), stripping jailbreak wrappers to the core imperative and comparing both TF-IDF cosine similarity and content-word Jaccard overlap. The per-HarmBench-behavior nearest-neighbor cosine similarity to any AdvBench training behavior averages 0.108 (median 0.124, max 0.442); no HarmBench behavior exceeds 0.50 cosine similarity to any AdvBench behavior, only one of 40 exceeds 0.30, and 15 of 40 are near-orthogonal (cosine < 0.05). The best content-word Jaccard overlap per HarmBench behavior averages 0.078, and 15 of 40 HarmBench behaviors share zero content words with any AdvBench training behavior. AdvBench carries no fine-grained topic labels, while HarmBench dev spans seven semantic categories; the most out-of-distribution HarmBench categories — verbatim copyright reproduction and misinformation edits — have no AdvBench analogue at all. The transfer is therefore evaluated on genuinely disjoint behaviors, consistent with a mechanistic account in which the trigger acts as a generic, content-addressed refusal-off switch and the frozen base model’s own latent capability supplies the harmful content, so transfer holds whenever training and evaluation goals share imperative structure.